AQI PREDICTOR

Multi-City Air Quality Forecasting

End-to-End MLOps System

PROJECT REPORT

Predicting the average AQI for the next 1, 2, and 3 days across five major Pakistani cities using automated data pipelines, feature engineering, model retraining, explainable AI, and a live dashboard.

Author
Muhammad Asif

10Pearls Shine Internship Program

September 2026

Executive Summary

The AQI Predictor is an end-to-end MLOps system designed to forecast the average Air Quality Index (AQI) for the next one, two, and three days across Islamabad, Karachi, Lahore, Multan, and Peshawar. Unlike a one-off machine-learning notebook, the system integrates live data collection, historical backfilling, feature engineering, a cloud feature store, automated model training and selection, model registration, SHAP explainability, CI/CD automation, automated testing, and a public Streamlit dashboard.

The reported experiments use more than 11,400 hourly observations at the time of reporting. Four model families—Ridge, Random Forest, XGBoost, and MLP—are compared against a naive persistence baseline. XGBoost provides the best results for one- and two-day forecasts, while Ridge performs best at the three-day horizon. The system also exposes model explanations and automatically raises hazard alerts when current or forecast AQI reaches 150 or above.

1. Introduction and Project Overview

Most air-quality applications report today's AQI. This project addresses a more demanding forecasting problem: estimating the average AQI for the next 1, 2, and 3 days across five major Pakistani cities. The objective is to build a production-oriented forecasting workflow rather than a static analytical notebook.

The implemented system provides:

- Hourly live data collection for AQI and weather across five cities.
- A Hopsworks Feature Store with three feature groups serving distinct purposes.
- Daily retraining in which four model families are compared automatically for each horizon.
- Automatic registration and selection of the best model using RMSE.
- SHAP-based explanations of the winning models.
- A live Streamlit dashboard with forecasts, trend visualization, model transparency, and hazard alerts.
- Automated pytest suites and post-training validation checks.

1.1 Problem Statement

Air-quality monitoring tools in Pakistan and elsewhere typically report only the current, instantaneous AQI reading. This is of limited use for planning purposes: schools, outdoor workers, athletes, and individuals with respiratory conditions need advance notice of deteriorating air quality, not just a snapshot of present conditions. Furthermore, air quality is highly city-specific, driven by local geography, industrial activity, and traffic patterns, so a single national forecast is insufficient. This project addresses the problem of producing short-horizon, per-city AQI forecasts that are reliable enough to support real decisions, generated through a system that keeps itself up to date without manual intervention.

1.2 Objectives

- Collect live and historical air-quality and weather data automatically for five major Pakistani cities.
- Engineer hourly predictive features and daily-average forecasting targets for 1-day, 2-day, and 3-day horizons.
- Train and compare multiple model families (Ridge, Random Forest, XGBoost, and MLP) against a naive persistence baseline, per forecast horizon.
- Automatically select and register the best-performing model per horizon through a cloud feature store and model registry, without manual version management.
- Provide model explainability through SHAP so that predictions are interpretable rather than opaque.
- Deploy a public-facing dashboard with automatic hazard alerting for unhealthy air-quality conditions.
- Establish automated testing and CI/CD scheduling so the system continuously retrains and refreshes itself with minimal manual oversight.

1.3 Methodology Overview

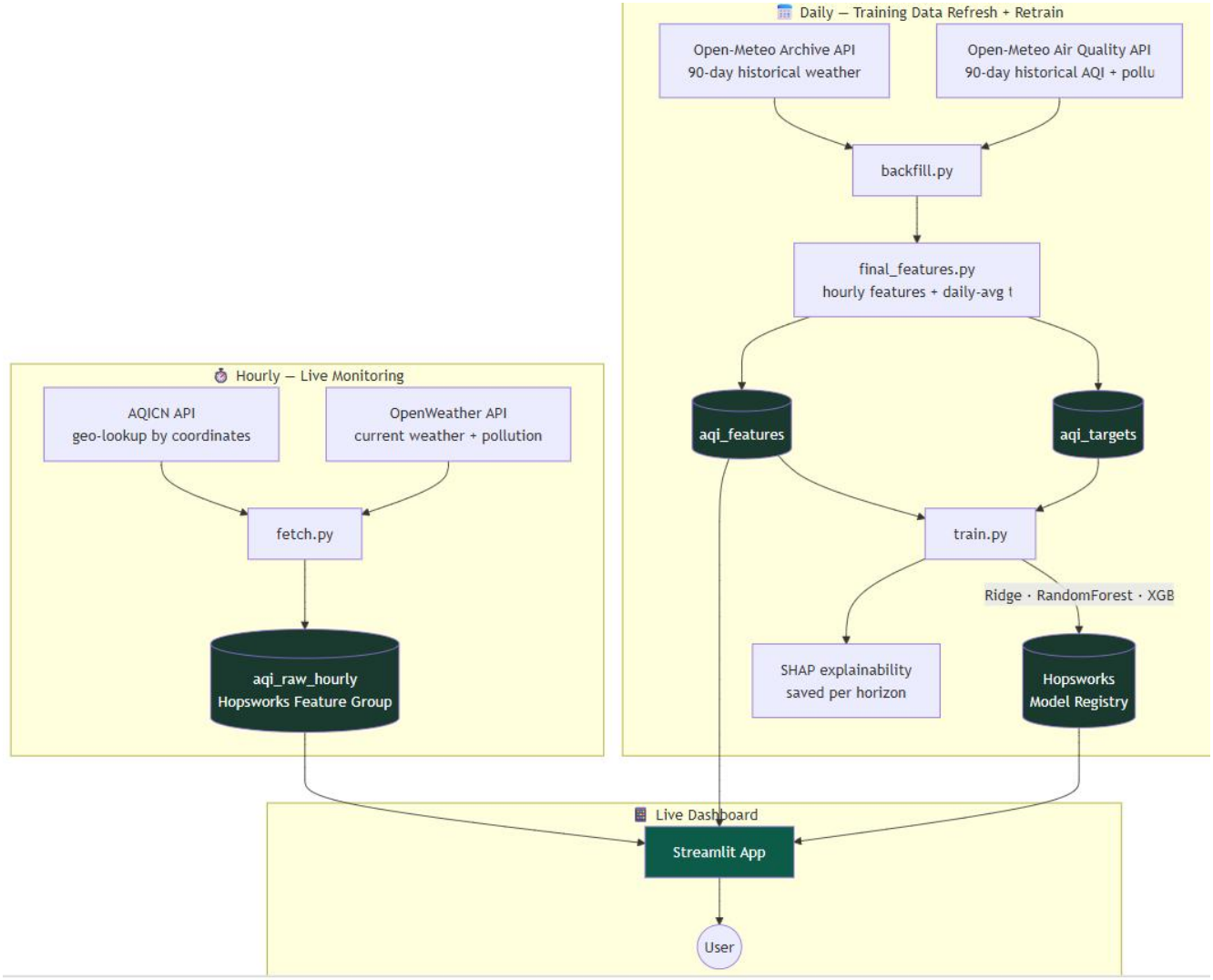
The project followed a structured, iterative MLOps methodology rather than a single-pass notebook workflow. The methodology consisted of the following stages: (1) live data collection from AQICN and OpenWeather on an hourly

schedule; (2) historical backfilling from Open-Meteo to obtain the depth of history required for lag and rolling features; (3) feature engineering to produce hourly predictors and daily-average targets, with explicit handling of missing hours; (4) storage of raw, feature, and target data in separate Hopsworks feature groups; (5) daily training and comparison of four model families per forecast horizon, evaluated by a calendar-date train/test split to avoid temporal leakage; (6) automatic registration of the best model per horizon by RMSE; (7) generation of SHAP explanations for the registered models; (8) deployment of a Streamlit dashboard reading live from the same feature store and model registry used in training; and (9) continuous validation through automated pytest suites and a dedicated automation-health check. Each stage was validated independently before the next was built on top of it, and real defects discovered during this process (see Section 13) were fixed and covered with regression tests rather than only patched ad hoc.

2. System Architecture

The architecture separates live monitoring, historical training-data preparation, and application serving. This separation reflects the different freshness and availability requirements of current observations and future prediction targets.

Live monitoring uses AQICN and OpenWeather, while the training pipeline uses Open-Meteo historical weather and air-quality data. Feature engineering creates hourly predictors and daily-average targets. These are stored separately in Hopsworks, where the training pipeline compares multiple models and registers the winner for each forecast horizon.



A key design decision is the use of two data paths. Live APIs provide genuinely current readings but do not provide the historical depth required for lag and rolling features. Open-Meteo's archive provides training history but has an approximately six-day latency. The dashboard therefore keeps the current AQI and forecast basis clearly separated.

3. Technology Stack

Layer	Technology
Live data	AQICN (WAQI) API, OpenWeather API
Historical data	Open-Meteo Historical Weather & Air Quality APIs
Geocoding	Open-Meteo Geocoding API
Feature Store	Hopsworks Serverless
ML Models	scikit-learn (Ridge/RidgeCV, Random Forest, MLPRegressor), XGBoost
Explainability	SHAP (Permutation Explainer)
Model Registry	Hopsworks Model Registry
Automation	GitHub Actions
Web Application	Streamlit + Plotly
Testing	pytest pipeline suites + post-training checks

4. Dataset and Exploratory Data Analysis

At the time of reporting, the dataset contains approximately 90+ days of hourly data for five cities, totaling more than 11,400 hourly rows and continuing to grow through automated refreshes.

City	Mean AQI	Std. Dev.	Min	Max	Character
Karachi	78.4	10.4	61	125	Consistently Moderate; least volatile
Islamabad	116.0	31.1	61	208	Moderate with meaningful daily swings
Peshawar	122.9	28.6	59	212	Similar profile to Islamabad
Multan	141.4	23.3	84	226	Consistently elevated
Lahore	148.8	37.7	74	364	Highest mean and highest volatility

The city profiles show a meaningful physical and geographic split. Karachi has the flattest AQI distribution, while Lahore has both the highest average and the greatest volatility, including the highest observed AQI.

4.1 Diurnal and Weekday Patterns

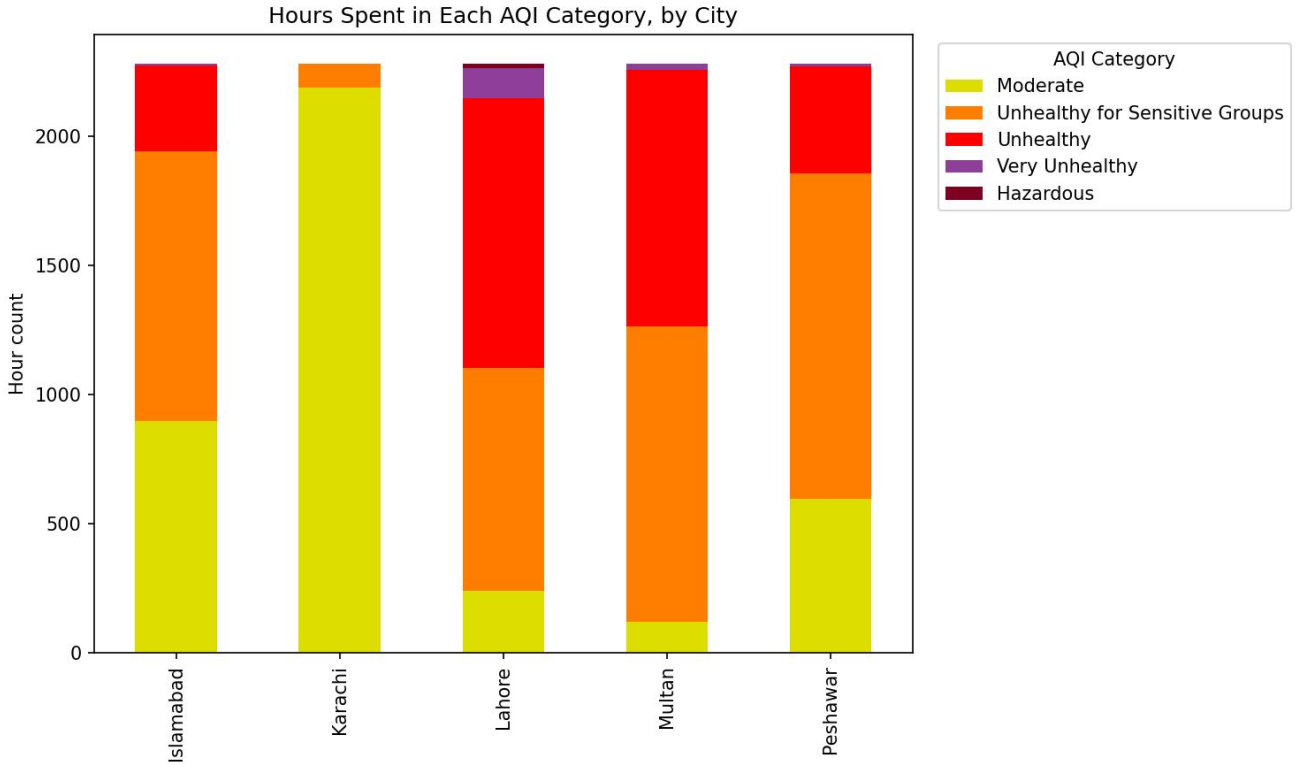
- Islamabad and Peshawar show a pronounced daytime increase, approximately from 100 to 155 between 9 AM and 2 PM local time.
- Lahore and Multan remain elevated throughout the day with a smaller midday increase, suggesting a stronger background pollution component.
- Karachi shows little evidence of a strong diurnal AQI cycle.
- Weekday differences are comparatively small, around 10–15 AQI points per city, while city and time-of-day differences are substantially larger.

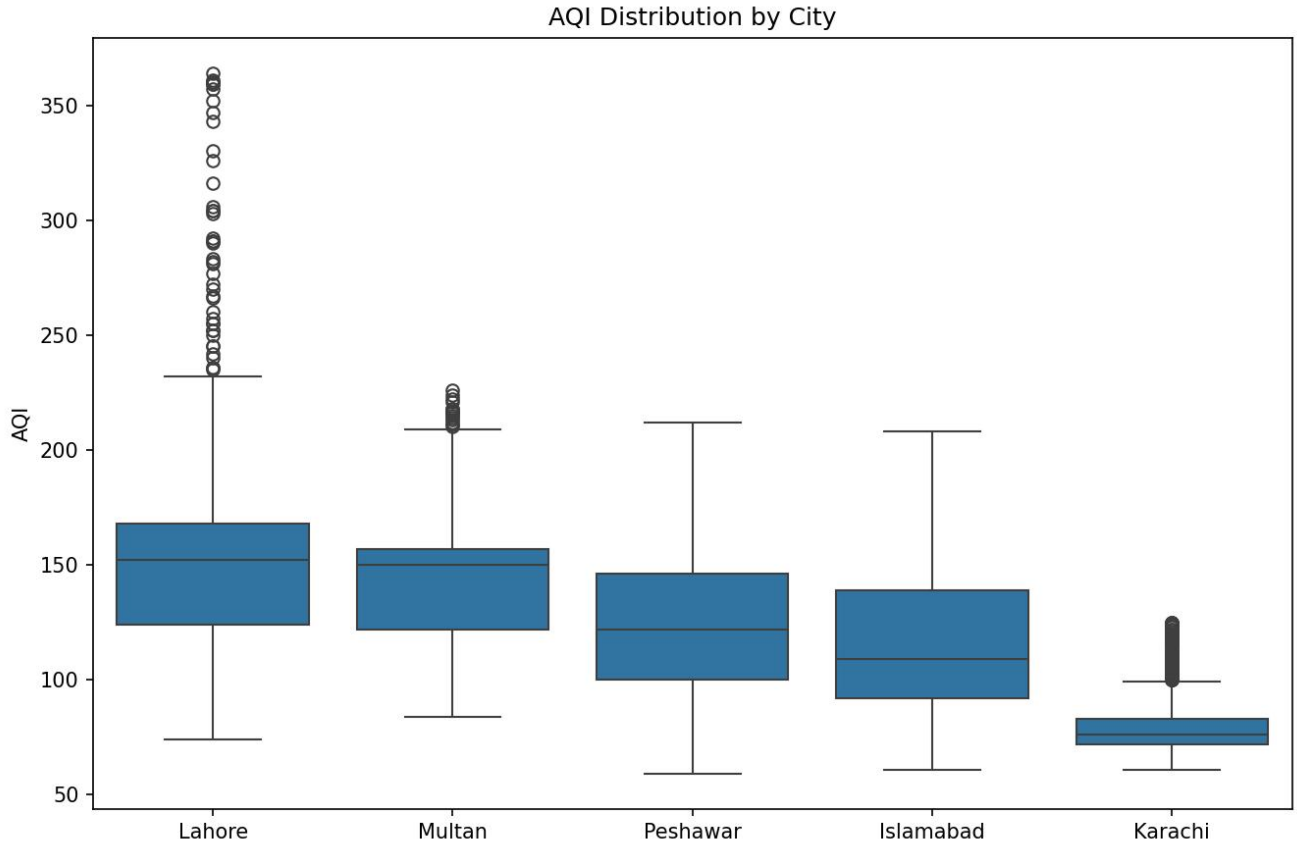
4.2 Cross-City Events and Correlation Checks

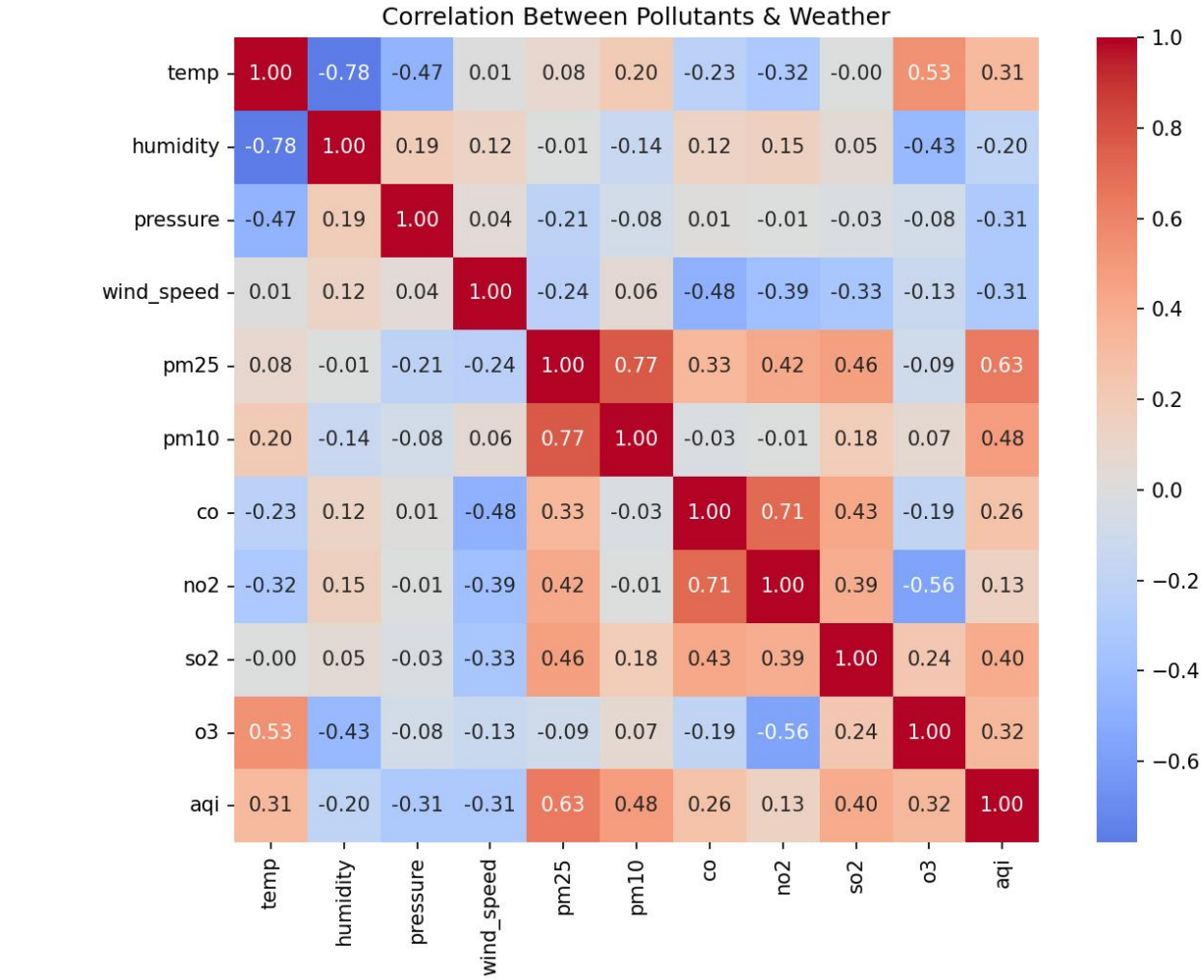
Two synchronized events were identified: a mid-June increase affecting Lahore and Multan simultaneously, and a late-July decline across all five cities. These patterns indicate that regional conditions can affect multiple cities at the same time.

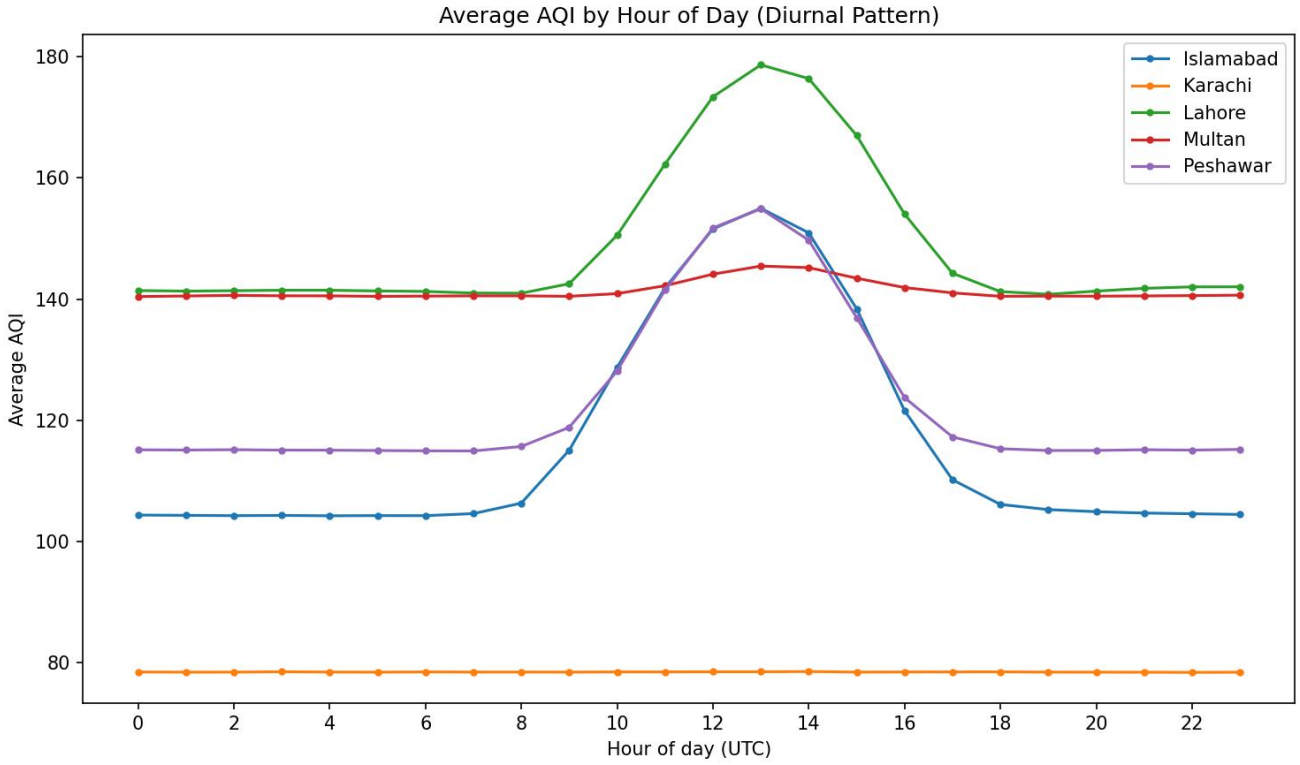
Relationship	Correlation	Interpretation
Temperature ↔ Humidity	−0.78	Expected inverse relationship
Wind speed ↔ CO / NO_2 / SO_2	−0.33 to −0.48	Wind dispersion reduces pollutant concentrations
CO ↔ NO_2	+0.71	Consistent with shared combustion/traffic sources
O_3 ↔ NO_2	−0.56	Consistent with NO_x titration of ozone
PM2.5 ↔ AQI	+0.63	AQI is based on the maximum pollutant sub-index, not PM2.5 alone

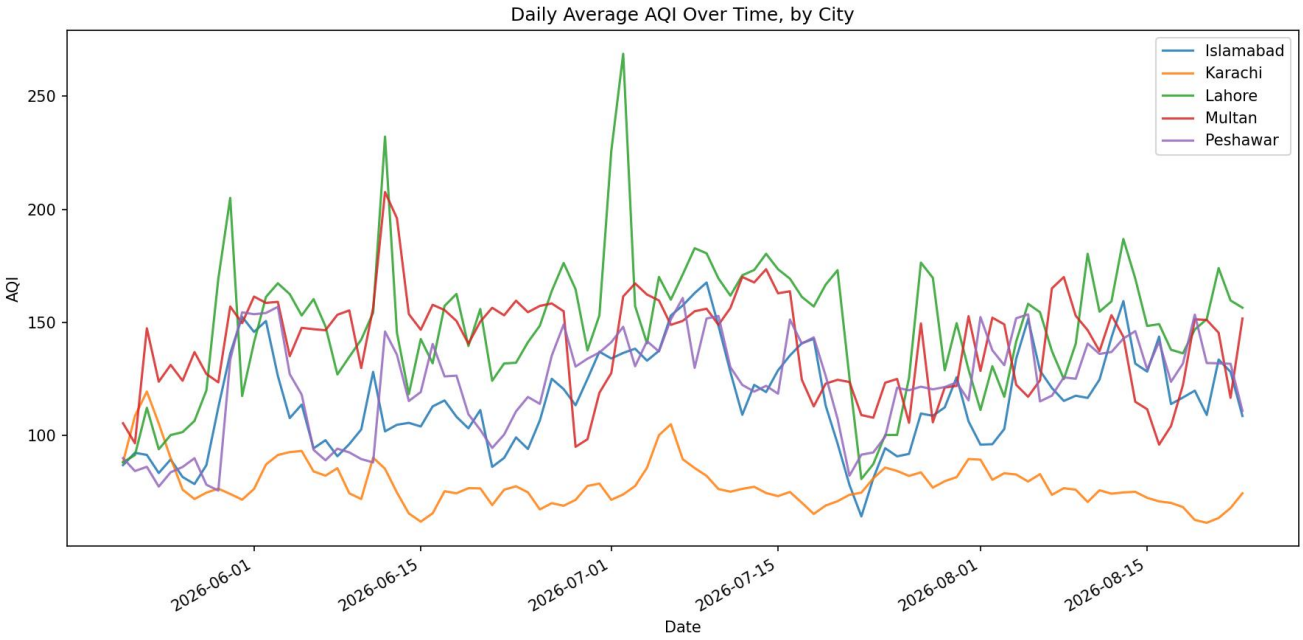
Missing values occur in lag and rolling feature columns and are expected warm-up artifacts created when a complete hourly grid is established before lag/rolling calculations. They are therefore treated as feature-construction effects rather than unexplained source-data loss.

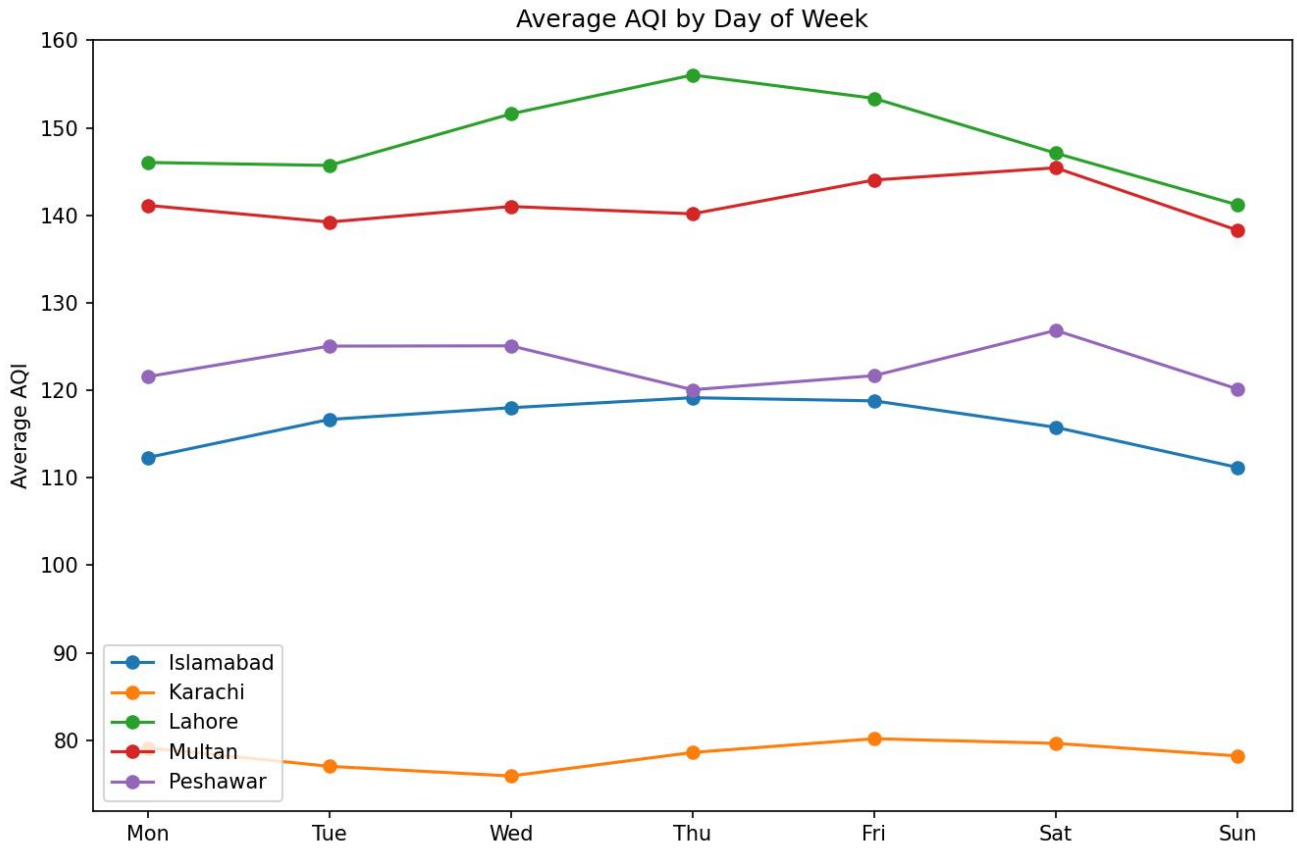

Hours Spent in Each AQI Category, by City
AQI Category
Moderate
Unhealthy for Sensitive Groups
Unhealthy
Very Unhealthy
Hazardous
Hour count
0
500
1000
1500
2000
Islamabad
Karachi
Lahore
Multan
Peshawar


AQI Distribution by City
AQI
50
100
150
200
250
300
350
Lahore
Multan
Peshawar
Islamabad
Karachi


Correlation Between Pollutants & Weather
temp 1.00 -0.78 -0.47 0.01 0.08 0.20 -0.23 -0.32 -0.00 0.53 0.31
humidity -0.78 1.00 0.19 0.12 -0.01 -0.14 0.12 0.15 0.05 -0.43 -0.20
pressure -0.47 0.19 1.00 0.04 -0.21 -0.08 0.01 -0.01 -0.03 -0.08 -0.31
wind_speed 0.01 0.12 0.04 1.00 -0.24 0.06 -0.48 -0.39 -0.33 -0.13 -0.31
pm25 0.08 -0.01 -0.21 -0.24 1.00 0.77 0.33 0.42 0.46 -0.09 0.63
pm10 0.20 -0.14 -0.08 0.06 0.77 1.00 -0.03 -0.01 0.18 0.07 0.48
co -0.23 0.12 0.01 -0.48 0.33 -0.03 1.00 0.71 0.43 -0.19 0.26
no2 -0.32 0.15 -0.01 -0.39 0.42 -0.01 0.71 1.00 0.39 -0.56 0.13
so2 -0.00 0.05 -0.03 -0.33 0.46 0.18 0.43 0.39 1.00 0.24 0.40
o3 0.53 -0.43 -0.08 -0.13 -0.09 0.07 -0.19 -0.56 0.24 1.00 0.32
aqi 0.31 -0.20 -0.31 -0.31 0.63 0.48 0.26 0.13 0.40 0.32 1.00
temp humidity pressure wind_speed pm25 pm10 co no2 so2 o3 aqi
1.0 0.8 0.6 0.4 0.2 0.0 −0.2 −0.4 −0.6


Average AQI by Hour of Day (Diurnal Pattern)
Islamabad
Karachi
Lahore
Multan
Peshawar
Average AQI
180
160
140
120
100
80
0 2 4 6 8 10 12 14 16 18 20 22
Hour of day (UTC)





5. Feature Engineering

The forecasting target is the average AQI for a calendar day one, two, or three days ahead. To preserve the project's required hourly time-based information while producing daily targets, a hybrid design is used: predictor features remain hourly, while daily-average targets are joined to hourly rows by calendar date.

Feature Group	Content	Granularity
Time	hour, day_of_week, month, is_weekend, sin/cos cyclical encodings	Hourly
Lag	aqi_lag_1h, aqi_lag_3h, aqi_lag_24h	Hourly
Rolling	24h rolling mean/std of AQI and PM2.5	Hourly
Derived	aqi_change_rate (gap-aware)	Hourly
Raw	temperature, humidity, pressure, wind, PM2.5/PM10, CO, NO_2, SO_2, O_3	Hourly
Categorical	city (one-hot encoded)	Model input
Targets	aqi_avg_next_1d / _2d / _3d	Daily, joined by date

A critical correctness safeguard is applied before lag and rolling features are computed: raw data is reindexed onto a complete hourly grid. Consequently, a missing hour becomes an explicit NaN instead of silently shifting the meaning of a lag such as 'one hour ago'.

6. Feature Store and Model Registry

Feature Group	Populated by	Frequency	Purpose
aqi_raw_hourly	fetch.py	Hourly	Live monitoring feed / current AQI
aqi_features	backfill.py	Daily	Model training inputs
aqi_targets	backfill.py	Daily	Daily-average AQI ground truth, 1/2/3 days ahead

The features and targets are intentionally stored separately. At prediction time, fresh features are available but future targets do not yet exist. This separation prevents the training target from being confused with available prediction-time information.

The application obtains the best registered model for each horizon using RMSE minimization, allowing the dashboard to automatically use the model that won the latest retraining cycle without manual version selection.

7. Model Training and Results

Four model families are trained and compared independently for each forecast horizon: Ridge, Random Forest, XGBoost, and MLP. A naive persistence baseline is also included to provide an honest reference point. The reported run uses 81 training days and 14 held-out test days, with the split performed by calendar date.

Horizon	Model	RMSE	MAE	R^2	Result
Tomorrow	Ridge	13.58	10.48	0.816	
Tomorrow	Random Forest	13.27	9.98	0.824	

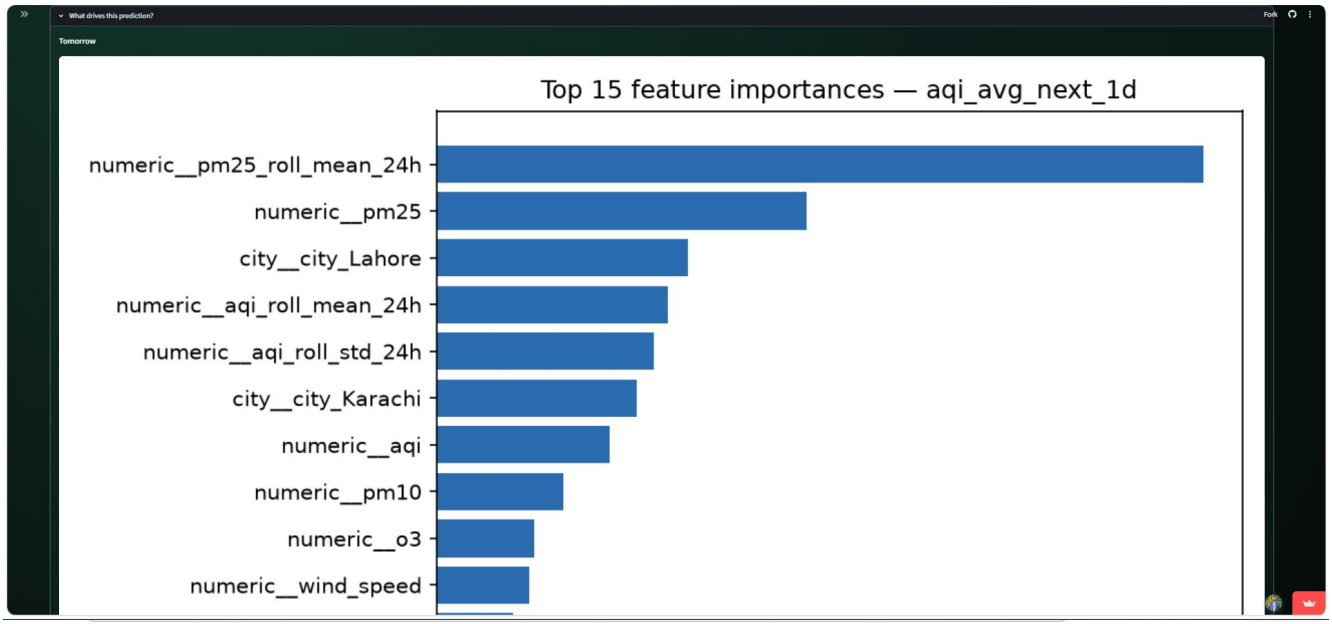
Tomorrow	XGBoost	12.82	9.81	0.836	Winner
Tomorrow	MLP	17.79	13.94	0.684	
Tomorrow	Naive baseline	16.37	12.68	0.732	Baseline
In 2 days	Ridge	16.24	12.37	0.743	
In 2 days	Random Forest	15.73	12.08	0.759	
In 2 days	XGBoost	15.61	12.42	0.762	Winner
In 2 days	MLP	23.85	18.73	0.445	
In 2 days	Naive baseline	19.01	14.37	0.647	Baseline
In 3 days	Ridge	16.32	12.89	0.740	Winner
In 3 days	Random Forest	16.63	12.95	0.730	
In 3 days	XGBoost	17.21	13.48	0.711	
In 3 days	MLP	21.84	17.02	0.534	
In 3 days	Naive baseline	21.82	16.15	0.535	Baseline

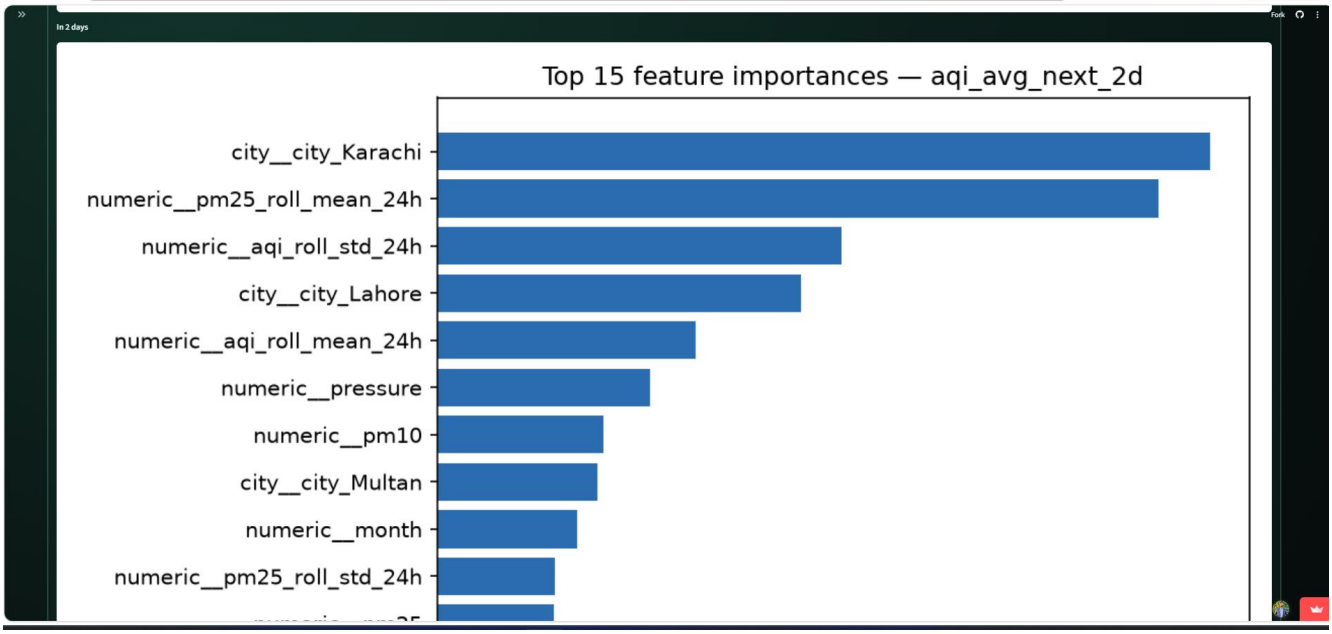
7.1 Key Findings

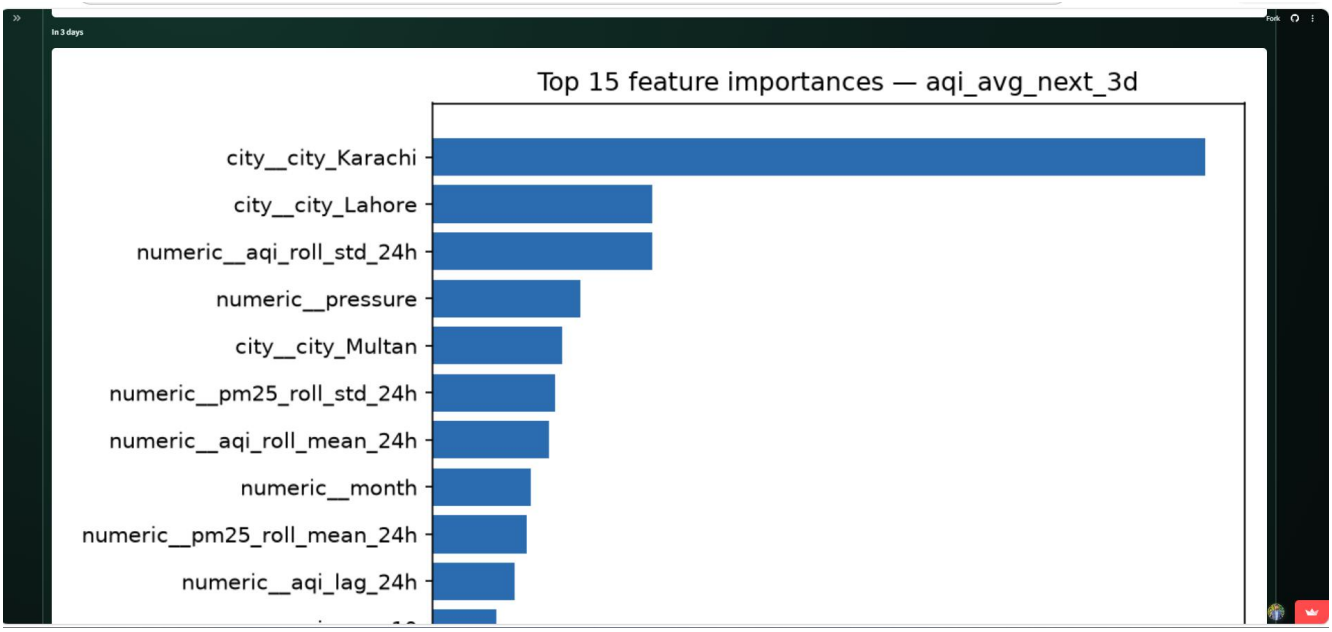
- Every evaluated model improves on the naive baseline at every forecast horizon.
- XGBoost is the strongest model at one and two days, whereas Ridge performs best at three days.
- The MLP underperforms the other models at all horizons. Its weaker performance is consistent with the relatively small number of independent training days available for a neural network.
- Pooling the five cities into a single multi-city model with city as a one-hot feature substantially improves model quality compared with training separate city-specific models.

8. Explainability with SHAP

SHAP Permutation Explainer is applied to the winning model for each forecast horizon. The explanations are stored as report-ready plots and surfaced in the dashboard's prediction-explanation panel.


What drives this prediction?
Fork
Tomorrow
Top 15 feature importances — aqi_avg_next_1d
numeric__pm25_roll_mean_24h
numeric__pm25
city__city_Lahore
numeric__aqi_roll_mean_24h
numeric__aqi_roll_std_24h
city__city_Karachi
numeric__aqi
numeric__pm10
numeric__o3
numeric__wind_speed


In 2 days
Fork
Top 15 feature importances — aqi_avg_next_2d
city__city_Karachi
numeric__pm25_roll_mean_24h
numeric__aqi_roll_std_24h
city__city_Lahore
numeric__aqi_roll_mean_24h
numeric__pressure
numeric__pm10
city__city_Multan
numeric__month
numeric__pm25_roll_std_24h



The explanations show a clear change with forecast horizon. Short-horizon predictions rely strongly on recent air-quality persistence, particularly rolling PM2.5 and AQI features. As the horizon increases, city identity, seasonality, and pressure become more important, indicating a greater reliance on climatological information as short-term persistence weakens.

9. CI/CD Automation

9.1 Hourly Feature Pipeline

The feature pipeline collects live AQI and weather data for all five cities and writes it to aqi_raw_hourly. Failures are isolated by city so a failed request for one city does not prevent the remaining cities from being stored. The workflow still reports the run as failed when a city request fails, making the problem visible in GitHub Actions.

9.2 Daily Training Pipeline

The daily workflow first refreshes the historical training window through backfill.py and then executes train.py. All four model families are retrained for each horizon, evaluated, and the winning models are registered.

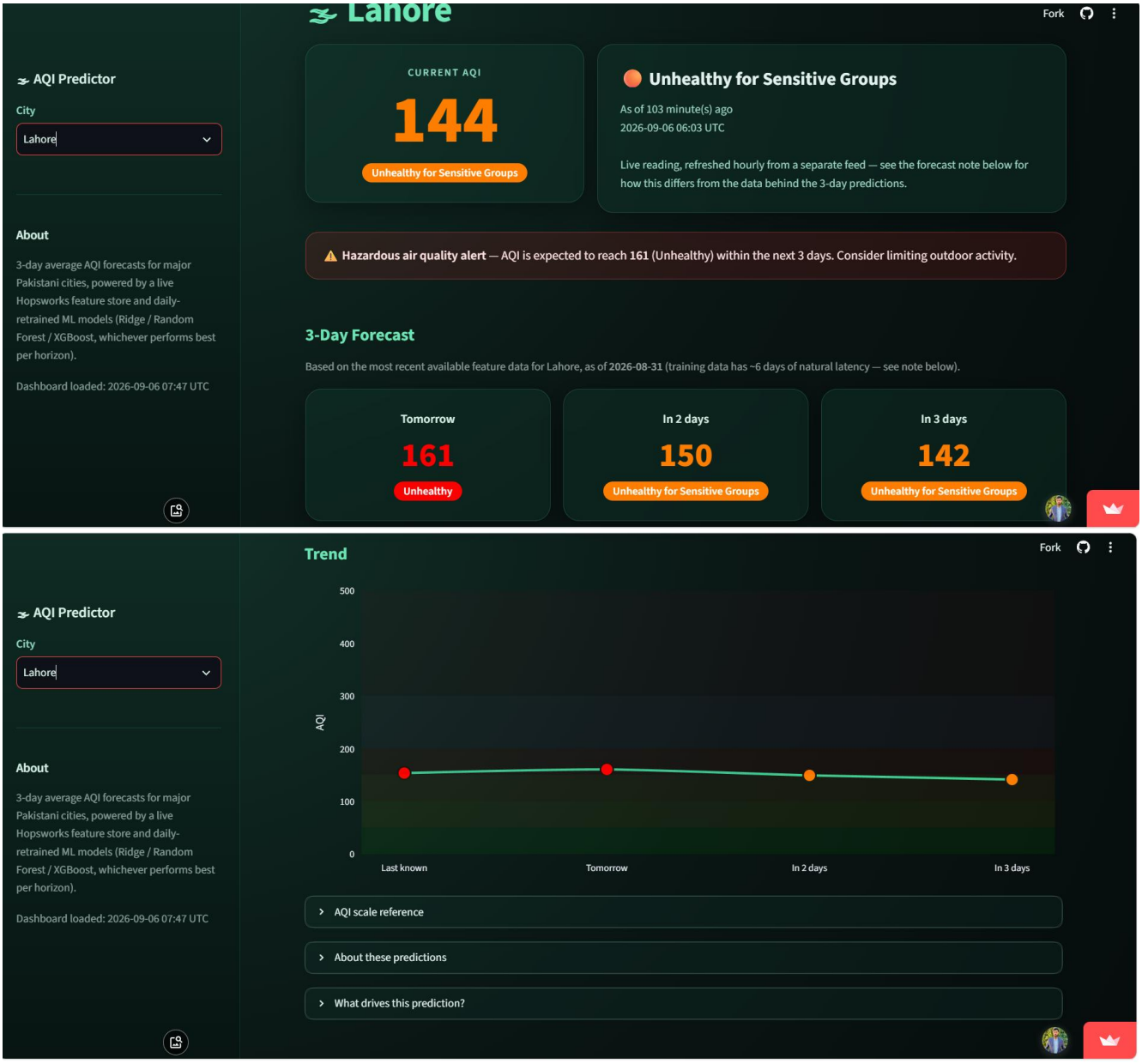
A platform limitation is documented: GitHub Actions scheduled workflows do not guarantee exact hourly execution intervals. The project includes check_automation_health.py so data freshness can be verified independently of the exact schedule.

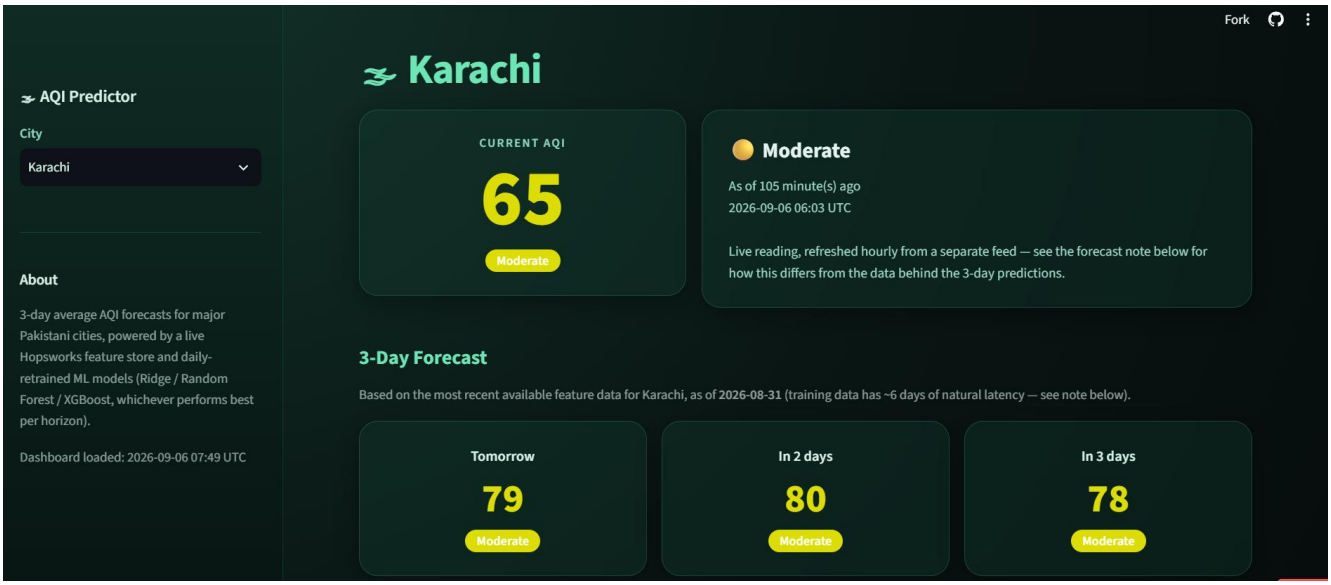
10. Web Application

The production dashboard is implemented using Streamlit and Plotly. It provides a city selector, live AQI, a three-day forecast, EPA AQI category visualization, an interactive trend chart, automatic hazard alerts, SHAP-based explanations, and model transparency information.

- City selector for Islamabad, Karachi, Lahore, Multan, and Peshawar.
- Live current AQI with a freshness indicator.
- Dated three-day forecast sourced from the feature store.
- AQI category color coding and interactive trend visualization.
- Automatic hazard alerting.
- Live 'What drives this prediction?' explanations.

- Model name, version, and test RMSE transparency.





11. Hazard Alerts

A hazard alert is triggered automatically when the current AQI or any forecast value reaches or exceeds 150, corresponding to the Unhealthy-and-above threshold used by the project. The alert logic is implemented as an independently tested is_hazardous() function.

Hazardous air quality alert — AQI is expected to reach 164 (Unhealthy) within the next 3 days.

Boundary testing verifies that AQI 150 triggers an alert while AQI 149 does not. Missing or NaN values are handled safely rather than causing failures.

12. Testing and Validation

The project includes more than 30 pytest tests across the feature and training pipelines, together with standalone post-training checks. Testing focuses on correctness of data ingestion, feature construction, leakage prevention, model evaluation, and prediction behavior.

Module	What's Tested
fetch.py	API parsing, retries/timeouts, multi-city partial-failure isolation, geo-based AQICN lookup
backfill.py	Historical parsing, schema validation, geocoding, city concatenation, date-range validation
features.py / daily_features.py / final_features.py	Lag/rolling correctness, gap handling, target alignment, daily-target joins
train.py	Date-based leakage prevention, metric calculations, winner selection, one-hot encoding, SHAP integration
test_predictions.py	Plausible ranges, baseline comparison, directional sensitivity, deterministic inference

The validation strategy therefore checks both individual pipeline components and the behavior of the final trained models.

13. Issues Identified and Resolved

- AQI category boundary bug: the original closed integer ranges created a gap for fractional predictions such as 150.4. The logic was changed to half-open intervals and six regression tests were added.
- AQICN station lookup failures: city-name lookup was unreliable for some cities, including Multan. The implementation was changed to coordinate-based geo lookup.
- Hopsworks feature-group version drift: version configuration was centralized through DEFAULT_FEATURE_GROUP_VERSION rather than duplicated across scripts.

14. Limitations and Design Decisions

1. Approximately six-day latency exists in the historical training features because of the source archive's availability/reanalysis delay.
2. GitHub Actions scheduling is not guaranteed to execute at exact hourly intervals.
3. The neural-network comparison uses MLPRegressor rather than a sequential LSTM or Transformer because the available independent training history is modest.
4. The live aqi_raw_hourly feature group is online-enabled for point lookups, while aqi_features and aqi_targets are intended for bulk training access.

15. Repository Structure

```
AQI-Predictor/
├── .github/workflows/
│   ├── feature_pipeline.yml
│   └── training_pipeline.yml
├── pipelines/
│   ├── feature_pipeline/
│   │   ├── fetch.py
│   │   ├── backfill.py
│   │   ├── features.py
│   │   ├── daily_features.py
│   │   ├── final_features.py
│   │   ├── get_coords.py
│   │   ├── hopsworks_io.py
│   │   ├── hopsworks_io2.py
│   │   ├── check_automation_health.py
│   │   ├── verify_hopsworks_upload.py
│   │   └── test_*.py
│   ├── training_pipeline/
│   │   ├── train.py
│   │   ├── test_train.py
│   │   ├── test_predictions.py
│   │   └── day6_shap_plots/
│   ├── app/
│   │   └── app.py
│   └── EDA/
│       ├── eda.py
│       └── eda_plots/
├── EDA/
│   └── EDA.py
├── experiment.ipynb
├── requirements.txt
├── LICENSE
└── README.md
```

16. Reproducibility and Execution

Prerequisites include Python 3.10 and free accounts/credentials for Hopsworks, AQICN, and OpenWeather.

```
git clone https://github.com/DataScienceWithAsif/AQI-Predictor.git
cd AQI-Predictor
pip install -r requirements.txt

Required environment variables:
OPENWEATHER_KEY=...
AQICN_TOKEN=...
HOPSWORKS_API_KEY=...
HOPSWORKS_PROJECT=AQI_Features
```

Pipeline execution:

```
python pipelines/feature_pipeline/backfill.py
python pipelines/training_pipeline/train.py
streamlit run pipelines/app/app.py
python pipelines/EDA/eda.py
python pipelines/feature_pipeline/check_automation_health.py

pytest pipelines/feature_pipeline/
pytest pipelines/training_pipeline/
```

17. Future Work

- Introduce a real sequential deep-learning model such as an LSTM after more historical data has accumulated.
- Extend the live monitoring stream into the training feature set through proper incremental feature engineering.
- Add additional Pakistani cities.
- Add push notifications for hazard alerts.

18. Data Sources and Acknowledgments

- Open-Meteo — historical weather and air-quality archive.
- AQICN / World Air Quality Index — live station-based AQI.
- OpenWeather — live weather and pollution data.
- Hopsworks — Feature Store and Model Registry.
- 10Pearls Shine Internship Program — project context.

Learning Outcomes and Reflection

Working on this project as part of the 10Pearls Shine Internship Program provided practical exposure to the full lifecycle of a production machine-learning system, going well beyond what a single modeling notebook can teach. Several concrete skills and lessons stand out from this experience.

- Building and operating a real feature store (Hopsworks), including the discipline of separating raw, feature, and target data rather than storing everything in one place.
- Integrating multiple third-party APIs (AQICN, OpenWeather, Open-Meteo) with different data freshness guarantees, and designing around their limitations (such as the roughly six-day historical latency) instead of ignoring them.
- Comparing model families rigorously against an honest baseline, and learning to accept a result that does not favor the most sophisticated model (the MLP) when the evidence does not support it.

- Debugging real, production-surfaced defects, such as the AQI category boundary bug and the AQICN station lookup failures, and converting each fix into a permanent regression test rather than a one-time patch.
- Setting up and troubleshooting CI/CD automation with GitHub Actions, including learning the practical limits of free-tier scheduled workflows rather than assuming cron schedules execute exactly as written.
- Communicating a technical system to a non-technical audience through the Streamlit dashboard and SHAP explanations, which reinforced the importance of interpretability alongside raw predictive accuracy.

Overall, the internship reinforced that production machine-learning work is primarily an engineering discipline: most of the effort and most of the value came from data reliability, testing, automation, and honest evaluation, rather than from model selection alone.

19. Conclusion

The AQI Predictor demonstrates a complete, automated MLOps workflow for multi-city air-quality forecasting. It combines data engineering, feature engineering, machine learning, model comparison, cloud feature management, explainability, automated testing, CI/CD, and a public-facing application in one reproducible system. The experimental results provide evidence that the selected approach adds predictive value over a persistence baseline and that the best model depends on forecast horizon. The project also makes its operational limitations explicit, including historical-data latency and scheduled-workflow precision, which strengthens its value as a production-oriented engineering project rather than a purely academic prototype.

Appendix A — Project Resources

Repository: github.com/DataScienceWithAsif/AQI-Predictor

Live dashboard: Live Dashboard